★ 구현 실습편 · II

멀티에이전트로 그룹웨어 직접 만들기

Claude Code와 revfactory/harness를 이용한 단계별 구현 가이드

비영리 법인 그룹웨어 사례를 초보자가 따라 만들 수 있게 프롬프트 중심으로 재구성했다

모든 단계는 같은 네 칸으로 진행한다 — 어디에, 무엇을 입력하고, 무엇이 나오며, 어떻게 피드백하는가.
코드는 한 줄도 직접 쓰지 않는다. 수강생은 붙여 넣고, 확인하고, 피드백한다.

이 가이드는 세 가지 원칙으로 움직인다

사례 I편이 무엇을 만들었는지 보여 줬다면, II편은 직접 만드는 길을 안내한다

1

한 명의 AI가 아니라 역할이 나뉜 팀이 만든다

기획, 개발, 회계 검수, 테스트를 맡은 에이전트들이 서로의 결과를 확인하며 일한다.

2

팀은 하네스가 설계하고, 사람은 개발 단계마다 확인한다

"하네스 구성해줘" 한 문장으로 팀이 생기고, 각 개발 단계의 결과는 사람이 보고 넘긴다.

3

실력은 프롬프트보다 피드백에서 갈린다

결과를 보고 무엇이 어디서 어떻게 달라야 하는지 말하는 것이 수강생의 핵심 역할이다.

수강생이 하는 일은 붙여 넣기, 확인, 피드백 세 가지뿐이다

멀티에이전트와 하네스의 뜻부터 정리한다

용어 세 개만 알면 이 가이드를 끝까지 따라올 수 있다

에이전트 (agent)

위임받아 완결하는 AI

역할과 규칙이 적힌 문서를 들고 일을 끝까지 해내는 Claude다. 질문에 답만 하는 비서와 다르다.

예) 회계 검수 에이전트

멀티에이전트 (multi-agent)

역할을 나눈 AI 팀

여러 에이전트가 작성과 검수를 나눠 맡는다. 한 명이 다 할 때보다 실수를 서로 잡아낸다.

작성–검수(writer–reviewer)

하네스 (harness)

팀을 움직이는 자동 실행 체계

누가 무엇을 언제 하는지 정한 에이전트 문서, 스킬, 진행 규칙의 묶음이다. 말의 마구에서 온 말이다.

하네스를 깎는다 = 개선한다

harness 플러그인은 팀 설계를 여덟 단계로 대신한다

revfactory/harness v2 README 기준(2026년 10월) — 우리는 결과만 확인한다

단계	harness가 하는 일
Phase 0	기존 하네스가 있는지 점검하고, 새로 만들지 고칠지 정한다
Phase 1	업무 영역을 분석하고 일의 흐름 모양을 파악한다
Phase 2	실행 방식과 팀 구조(아키텍처 패턴)를 고른다
Phase 3~4	에이전트 문서와 스킬 문서를 만든다
Phase 5	오케스트레이터를 만들고 CLAUDE.md에 실행 규칙을 적는다
Phase 6	만든 팀이 실제로 동작하는지 시험한다
Phase 7	쓰고 난 뒤 피드백으로 팀을 고친다 (/harness:evolve)

팀 구조는 여섯 패턴 가운데 일에 맞는 것을 고른다

harness가 지원하는 아키텍처 패턴과 이 프로젝트에서 쓰는 자리

패턴	모양	이 프로젝트에서 쓰는 곳
Pipeline	앞 단계 결과가 다음 단계 입력이 된다	설계 → 구현 → 검증 → 배포
Fan-out/Fan-in	여러 일을 동시에 맡기고 모은다	인사·사업·게시판·AI 모듈 병렬 구현
Producer-Reviewer	작성한 것을 다른 에이전트가 검수한다	회계 전표 검수, 화면 테스트
Supervisor	감독자가 일을 나누고 결과를 확인한다	오케스트레이터의 단계 관리
Expert Pool	필요할 때 전문가를 골라 부른다	양식 재현, 배포 같은 특수 작업
Hierarchical	위에서 아래로 단계별 위임한다	이번 규모에서는 쓰지 않는다

모든 실습 슬라이드는 같은 네 칸으로 되어 있다

어디 → 입력 → 출력 → 피드백 순서로 읽고, 그대로 따라 한다

어디	[입력]	[출력]	[피드백]
어느 창에 입력하는가	**붙여 넣을 프롬프트**	**Claude가 보여 줄 결과**	**결과를 보고 다시 보내는 말**
터미널인지, Claude Code 입력창인지, 파일 탐색기인지 먼저 확인한다.	앰버 상자 안의 글을 그대로 복사해 붙여 넣는다. 고칠 곳은 우리 법인 정보뿐이다.	비슷한 결과가 나오면 성공이다. 숫자와 파일 이름은 조금 달라도 된다.	예시처럼 무엇이 어디서 어떻게 다른지 구체적으로 말한다.

단계가 끝날 때마다 "지금 상태를 기록 저장해 줘"라고 입력한다 (git 커밋)

열세 단계를 지나면 그룹웨어가 완성된다

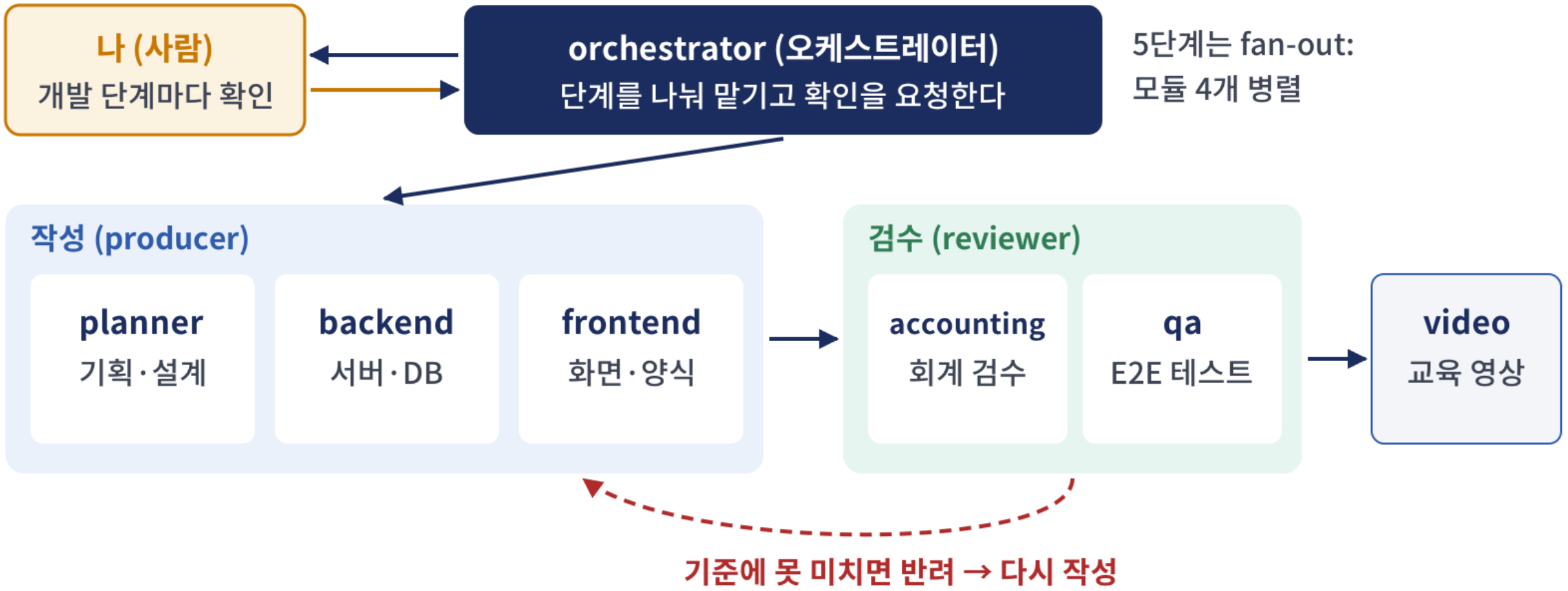
준비 세 단계, 팀 설계 두 단계, 구현 다섯 단계, 검증·배포·회고 세 단계



모든 단계는 같은 네 칸으로 진행한다: 어디 → 입력 → 출력 → 피드백. 단계가 끝나면 기록 저장(git)한다.

작성 팀과 검수 팀을 나누고 사람이 확인한다

이 가이드가 만드는 에이전트 팀 — 이름은 영어, 역할은 한국어로 적는다



01

준비 · STEP 0~2

작업장과 재료를 준비한다

Claude Code 설치, 업무 자료 정리, harness 플러그인 설치

STEP 0

Claude Code를 설치하고 빈 프로젝트 폴더에서 시작한다

사람이 직접 하는 유일한 설치 단계다 — 이후는 모두 Claude Code에게 시킨다

어디 — 터미널 — Mac은 '터미널' 앱, Windows는 'PowerShell'

[입력] 명령어 · 터미널에 한 줄씩

```
# 1) 설치 (Mac·Linux)
curl -fsSL https://claude.ai/install.sh | bash

# 1) 설치 (Windows PowerShell)
irm https://claude.ai/install.ps1 | iex

# 2) 새 터미널 창을 열고 확인
claude --version

# 3) 프로젝트 폴더를 만들고 시작
mkdir groupware
cd groupware
claude
```

[출력] Claude가 보여 주는 결과

```
버전 번호가 한 줄 나온다
  예) 2.x.x (Claude Code)

처음 실행하면 로그인 안내가 뜬다
→ Claude Max 계정으로 로그인
```

[피드백] 결과를 보고 같은 입력창에 다시 보낸다

```
'claude'를 찾을 수 없다고 나오면
터미널 창을 새로 열고 다시 입력한다.
그래도 안 되면 오류 문장을 복사해
Claude 채팅창에 붙여 넣고 물어본다.
```

STEP 1

업무 자료를 모아 Claude에게 요구사항을 정리시킨다

먼저 groupware 폴더 안에 docs 폴더를 만들고 양식·규정·계정과목표를 넣어 둔다

어디 — **Claude Code 입력창 — groupware 폴더에서 실행 중**

[입력] 프롬프트 · Claude Code 입력창

```
docs 폴더에 우리 법인의 업무 자료를 넣어 두었다.
품의서·지출결의서·급여결의서 양식(HWP),
결재 규정, 계정과목표, 회계프로그램에서
내보낸 CSV 샘플이 들어 있다.

1. 모든 파일을 읽어라. HWP는 텍스트로 바꿔 읽어라.
2. 업무를 requirements.md로 정리해라.
   - 업무 목록, 담당자, 결재선
   - 양식별 입력 칸
   - 결재가 끝난 뒤 회계에 생겨야 할 기록
3. 자료로 알 수 없는 것은 질문 목록에 적어라.
   추측해서 채우지 마라.
```

[출력] Claude가 보여 주는 결과

```
requirements.md 생성
업무 12개 (결재 5, 회계 4, 인사 3)
결재선: 담당 → 사무국장 → 이사장
질문 4개
 - 10만 원 이하 지출은 전결인가?
 - 후원금 영수증은 누가 발급하나?
```

[피드백] 결과를 보고 같은 입력창에 다시 보낸다

```
질문에 답한다.
10만 원 이하는 사무국장 전결이다.
후원금 영수증은 사무국이 발급한다.
답을 requirements.md에 반영해 줘.
```

STEP 2

harness 플러그인은 슬래시 명령 두 줄로 설치한다

v2부터는 실험 기능 환경 변수가 필요 없다 (README 기준)

어디 — **Claude Code 입력창 — 슬래시(/)로 시작하는 명령**

[입력] 명령어 · Claude Code 입력창

```
/plugin marketplace add revfactory/harness
/plugin install harness@harness-marketplace

# 설치가 끝나면 Claude Code를 다시 연다
/exit
claude

# 설치 확인
/plugin
```

[출력] Claude가 보여 주는 결과

```
마켓플레이스 추가 완료
harness 플러그인 설치 완료

/plugin 목록에 harness가 보인다
```

[피드백] 결과를 보고 같은 입력창에 다시 보낸다

```
오류가 나면 메시지를 그대로 붙여 넣고
다음처럼 입력한다.

이 오류를 해결하고 harness 설치를
끝까지 마쳐 줘.
```

02

팀 설계 · STEP 3~4

에이전트 팀을 만들고 설계를 확인한다

"하네스 구성해줘" 한 문장으로 팀을 만들고, 설계 문서를 사람이 검토한다

STEP 3

"하네스 구성해줘" 한 문장이 에이전트 팀을 만든다

조건을 함께 적으면 harness가 패턴과 역할을 그에 맞춰 정한다

어디 Claude Code 입력창

[입력] 프롬프트 · Claude Code 입력창

```
하네스 구성해줘.

만들 것: 사단법인용 올인원 그룹웨어 웹앱
근거 자료: requirements.md를 먼저 읽어라.

팀 구성 조건
- 기획 에이전트가 설계 문서를 쓴다
- 백엔드·프론트엔드 에이전트가 구현한다
- 회계 검수 에이전트가 전표와 CSV를 따로 검사한다
- QA 에이전트가 브라우저로 E2E 테스트를 한다
- 작성-검수(producer-reviewer) 구조로 묶어라

단계가 끝날 때마다 나에게 확인받고 넘어가라.
```

[출력] Claude가 보여 주는 결과

```
.claude/agents/
  planner  backend  frontend
  accounting-reviewer  qa
.claude/skills/
  groupware-orchestrator  ledger-export ...
CLAUDE.md (실행 규칙, 변경 이력)
```

[피드백] 결과를 보고 같은 입력창에 다시 보낸다

```
교육 영상 에이전트를 하나 더 둬 줘.
QA가 녹화한 테스트에 자막을 붙여
사용법 영상을 만드는 역할이다.
```

STEP 3

에이전트 파일은 한국어로 고칠 수 있는 업무 분장표다

만들어진 파일 하나를 열어 읽는 법을 익힌다

[출력] 예시 · .claude/agents/accounting-reviewer.md

```
---
name: accounting-reviewer
description: 전표와 CSV를 검수한다.
  backend가 회계 코드를 바꿀 때마다 부른다.
---

## 확인할 것
1. 승인 문서 1건에 전표가 1건인가
2. 차변 합계와 대변 합계가 같은가
3. CSV가 장부 프로그램 형식과 같은가

## 하지 않을 것
- 장부 정본을 직접 고치지 않는다
- 기준을 못 넘으면 backend에 반려한다
```

읽는 법

- description은 이 에이전트를 언제 부르는지 정한다
- 확인할 것이 곧 검수 기준이다
- 하지 않을 것이 가드레일이다
- 문장을 고치면 팀의 일하는 방식이 바뀐다

STEP 4

설계 문서를 먼저 만들고 사람이 확인한 뒤 구현한다

개발 단계의 사람 확인(HITL)이다 — 품의서의 업무 결재와는 별개다

어디 **Claude Code 입력창**

[입력] 프롬프트 · Claude Code 입력창

```
오케스트레이터로 1단계(설계)만 실행해 줘.

기획 에이전트가 만들 문서
- spec.md: 화면 목록과 기능
- data-model.md: 테이블과 관계
- events.md: 결재 상태와 회계 연동 규칙

꼭 지킬 규칙
- 품의서 승인 → 집행 예약 생성
- 지출결의서 승인 → 전표 생성
- 장부 정본은 외부 회계프로그램이다.
  우리는 그 형식의 CSV만 낸다.

문서가 다 되면 요약을 보여 주고 멈춰라.
```

[출력] Claude가 보여 주는 결과

```
설계 완료 — 확인 대기
화면 24개, 테이블 15개
결재 상태: 임시저장 → 진행 → 승인·반려
이벤트: 품의 승인, 지출결의 승인
다음 단계로 갈까요?
```

[피드백] 결과를 보고 같은 입력창에 다시 보낸다

```
넘어가기 전에 두 가지를 고쳐 줘.
1. 전결 규칙(10만 원 이하)을 events.md에 넣어라.
2. 승인 뒤 수정은 소급·정정 이력으로 남겨라.
고친 뒤 다시 요약해 줘.
```

03

구현 · STEP 5~9

모듈을 하나씩, 때로는 동시에 만든다

기반 → 결재 → 회계 → 통장 분류 → 부가 모듈 순서로 쌓는다

STEP 5

로그인과 역할 권한이 모든 모듈의 바닥이 된다

기술 선택은 에이전트에게 맡기되 이유를 기록하게 한다

어디 Claude Code 입력창

[입력] 프롬프트 · Claude Code 입력창

```
설계를 확인했다. 2단계(기반)를 진행해 줘.

- 기술 선택은 기획 에이전트가 정하되
  이유를 README에 한 줄씩 적어라
- 로그인, 조직, 사용자, 역할 권한을 만든다
- 역할: 직원, 사무국장, 이사장, 관리자
- 모든 변경은 감사로그에 남긴다
- 테스트 계정 4개를 seed 파일에 만든다

끝나면 로컬 주소를 알려 주고 화면을 열어 줘.
```

[출력] Claude가 보여 주는 결과

```
기반 완료 · http://localhost:5165
테스트 계정 4개 → seed/users.json
감사로그 테이블 생성
QA: 로그인·권한 테스트 12개 통과
```

[피드백] 결과를 보고 같은 입력창에 다시 보낸다

```
직원 계정으로 들어갔는데 관리자 메뉴가 보인다.
권한을 고치고, 이 경우를 테스트에 추가해 줘.
```

STEP 6

결재 엔진을 만들고 양식은 원본과 나란히 비교한다

"기존 양식 그대로"를 확인하는 가장 빠른 방법은 비교 캡처다

어디 Claude Code 입력창

[입력] 프롬프트 · Claude Code 입력창

```
3단계(결재)를 진행해 줘.

1. events.md의 상태대로 결재 엔진을 만든다.
2. docs의 품의서 양식을 HTML로 똑같이 그린다.
   결재란 칸은 결재선 데이터로 만든다.
3. 인쇄하면 A4 한 장에 원본처럼 나와야 한다.
4. 원본과 출력물을 나란히 캡처해 보여 줘.

회계 연동은 아직 하지 마라.
```

[출력] Claude가 보여 주는 결과

```
결재 엔진 완료 (상태 4개, 전결 규칙 1개)
양식 3종: 품의서, 지출결의서, 급여결의서
compare/품의서_원본_대_출력.png
QA: 기안·승인·반려 시나리오 통과
```

[피드백] 결과를 보고 같은 입력창에 다시 보낸다

```
출력물 결재란이 원본보다 좁고
'관련 근거' 칸이 빠졌다.
원본과 같게 고치고 비교 캡처를 다시 보여 줘.
```

STEP 7

승인 이벤트를 전표로 잇고 회계 검수 에이전트가 확인한다

작성–검수 구조가 가장 큰 힘을 내는 단계다

어디 Claude Code 입력창

[입력] 프롬프트 · Claude Code 입력창

```
4단계(회계 연동)를 진행해 줘.

- 품의서 승인 시 집행 예약을 만든다
- 지출결의서 승인 시 전표를 만든다
- 계정 대응표로 우리 과목을 장부 과목에 맞춘다
- 전표를 docs의 CSV 샘플과 같은 형식으로 낸다

accounting-reviewer가 가짜 문서 30건으로 검수하고
통과하지 못하면 backend에 돌려보내라.
검수 결과를 표로 보여 줘.
```

[출력] Claude가 보여 주는 결과

```
회계 검수 결과 (가짜 문서 30건)
전표 1:1 대응          30/30 통과
차대 합계 일치         30/30 통과
CSV 형식 일치          날짜 형식 달라 반려
재작업 후 재검수       통과
```

[피드백] 결과를 보고 같은 입력창에 다시 보낸다

```
CSV를 회계프로그램에 실제로 올려 보니
'부가세 칸이 비어 있다'는 오류가 났다.
오류 화면을 붙여 넣는다. 원인을 찾아 고쳐 줘.
```

STEP 8

통장 거래 분류는 별도 도구로 만들고 반영은 사람이 한다

실제 통장 파일 대신 가짜 거래로 먼저 시험한다

어디 **Claude Code 입력창**

[입력] 프롬프트 · Claude Code 입력창

```
통장 거래 분류 도구를 따로 만들어 줘.
이름은 bank-classifier로 한다.

- inbox 폴더의 통장·CMS 엑셀을 읽는다
- 거래마다 계정과목을 붙인다
- 후원금은 회원 이름 없이 세목별로 합친다
- 같은 거래는 두 번 올리지 않는다
- 결과를 JSON으로 그룹웨어에 올린다

가짜 거래 100건을 만들어 시험해라.
반영은 사람이 미리보기 화면에서 확정한다.
```

[출력] Claude가 보여 주는 결과

```
가짜 거래 100건
→ 분류 96건, 확인 필요 4건
중복 업로드 시험: 추가 0건 (정상)
그룹웨어 '미리보기 대기'에 1묶음 표시
```

[피드백] 결과를 보고 같은 입력창에 다시 보낸다

```
확인 필요 4건은 모두 이체 수수료다.
수수료는 '지급수수료' 과목으로
분류하도록 규칙에 추가해 줘.
```

STEP 9

서로 독립된 모듈은 fan-out으로 동시에 만든다

병렬로 만들고, 합친 뒤에는 전체 회귀 테스트로 확인한다

어디 — **Claude Code 입력창**

[입력] 프롬프트 · Claude Code 입력창

```
5단계(부가 모듈)를 병렬로 진행해 줘.
fan-out 방식으로 네 모듈을 동시에 맡겨라.

A. 인사·근태: 연차, 출퇴근
B. 사업 관리: 사회복지 사업 실적 기록
C. 게시판·알림
D. AI 도우미: 기안 초안 작성, 규정 질문 답변

모듈마다 QA 테스트를 통과해야 합친다.
합친 뒤 전체 회귀 테스트를 돌려라.
```

[출력] Claude가 보여 주는 결과

```
병렬 작업 4건 완료
A 통과 · B 통과 · C 통과
D 보류 (규정 근거 없이 답함)
회귀 테스트 186개 통과
```

[피드백] 결과를 보고 같은 입력창에 다시 보낸다

```
D는 docs/규정 폴더만 근거로 답하게 해.
근거를 못 찾으면 '규정에서 찾지 못했다'고
답하게 고쳐 줘.
```

04

검증·배포·회고 · STEP 10~12

검증하고, 올리고, 팀을 고친다

E2E 테스트와 교육 영상, Azure 배포, 하네스 회고

STEP 10

QA가 브라우저로 시험하고 그 녹화가 교육 영상이 된다

사례 I편 캡처의 자동화 경고 배너가 바로 이 단계의 흔적이다

어디	Claude Code 입력창

[입력] 프롬프트 · Claude Code 입력창

```
6단계(검증)를 진행해 줘.

QA 에이전트가 브라우저로 직접 시험한다.
시나리오: 직원 기안 → 사무국장 검토
→ 이사장 승인 → 전표 생성 → CSV 내보내기

과정을 녹화하고, 교육 영상 에이전트가
단계마다 한국어 자막을 붙여
사용법 영상으로 만들어라.
```

[출력] Claude가 보여 주는 결과

```
E2E 시나리오 8개 통과
videos/결재_사용법.mp4 (3분 20초)
videos/통장거래_올리기.mp4 (2분 10초)
```

[피드백] 결과를 보고 같은 입력창에 다시 보낸다

```
영상이 너무 빠르다.
클릭마다 1초씩 멈추고
자막 글씨를 키워서 다시 만들어 줘.
```

STEP 11

배포는 계획서를 먼저 확인받은 뒤 실행한다

MS 비영리 프로그램 Azure 크레딧 사용 — 클라우드 로그인은 사람이 직접 한다

어디 Claude Code 입력창

[입력] 프롬프트 · Claude Code 입력창

```
7단계(배포)를 준비해 줘. Azure에 올린다.

- 배포 계획을 deploy-plan.md로 먼저 써라
  (App Service, PostgreSQL, Blob Storage, Key Vault)
- 비밀값은 코드에 넣지 말고 Key Vault에 둔다
- 로그인은 MS365 계정(Entra ID)으로 바꾼다
- 매일 백업하고 복구 절차를 문서로 남긴다

Azure 로그인은 내가 직접 한다.
계획을 보여 주고 확인받은 뒤 실행해라.
```

[출력] Claude가 보여 주는 결과

```
deploy-plan.md — 리소스 6개, 예상 월 비용
확인 대기

(확인 후) 배포 주소 안내
  https://(법인명)-groupware.azurewebsites.net
```

[피드백] 결과를 보고 같은 입력창에 다시 보낸다

```
비영리 크레딧 한도 안에 들도록
가장 작은 사양으로 줄이고
비용을 다시 계산해 줘.
```

STEP 12

회고로 반복된 문제를 팀 규칙에 새겨 넣는다

harness의 진화 단계(/harness:evolve) — 하네스를 깎는다

어디	Claude Code 입력창

[입력] 프롬프트 · Claude Code 입력창

```
하네스 회고해줘.

이번 프로젝트에서 반복된 문제
- 양식 출력이 원본과 달라 세 번 고쳤다
- 회계 CSV 형식 오류가 두 번 났다

같은 문제가 다시 생기지 않도록
에이전트와 스킬을 고쳐라.
바꾼 내용을 CLAUDE.md 변경 이력에 남겨라.
```

[출력] Claude가 보여 주는 결과

```
변경 3건
- frontend: 원본 비교 캡처를 필수 단계로
- accounting-reviewer: CSV 실제 업로드 시험 추가
- CLAUDE.md 변경 이력 v1.1
```

[피드백] 결과를 보고 같은 입력창에 다시 보낸다

```
좋다. 다음 달 연말정산 기능도
같은 하네스로 진행할 수 있게 준비해 줘.
```

좋은 피드백은 무엇이 어디서 어떻게 달라야 하는지 말한다

막연한 말은 에이전트를 헤매게 하고, 구체적인 말은 한 번에 고치게 한다

막연한 피드백	구체적인 피드백
이상해요	결재란이 원본보다 좁다. 원본처럼 세 칸을 같은 폭으로 맞춰 줘
다시 해 줘	CSV 날짜가 2026/11/30 형식이다. 2026-11-30 형식으로 바꿔 줘
알아서 고쳐	오류 화면을 붙여 넣는다. 원인을 먼저 설명하고 고쳐 줘
전부 바꿔	회계 모듈만 고치고 다른 모듈은 건드리지 마

피드백이 두 번 이상 반복되면 STEP 12처럼 에이전트 규칙으로 옮긴다

막히면 증상을 그대로 말하는 것이 가장 빠른 해결책이다

자주 생기는 다섯 가지 상황과 그대로 붙여 넣을 문장

이런 증상이 보이면	이렇게 입력한다
에이전트끼리 같은 파일을 고쳐 충돌한다	충돌을 정리하고, 모듈마다 담당 폴더를 나눠 줘
작업이 길어져 앞의 결정을 잊는다	지금까지의 결정을 progress.md에 정리하고 이어서 해 줘
테스트가 계속 실패한다	실패한 테스트 하나만 골라 원인을 설명해 줘
무엇이 바뀌었는지 모르겠다	오늘 바뀐 것을 비전공자가 알 수 있게 다섯 줄로 요약해 줘
어제 잘 되던 것이 망가졌다	마지막으로 잘 되던 기록 저장 시점으로 되돌려 줘

실제 직원 데이터를 넣기 전에 세 가지를 확인한다

가짜 데이터로 끝까지 시험한 뒤에만 실제 데이터를 넣는다

개인정보 보호

급여, 인사, 복지 대상자 정보가 암호화되는지, 권한 없는 계정으로 열리지 않는지 QA에게 시험시킨다.

백업과 복구

백업 파일로 실제 복구가 되는지 한 번 시연한다. 복구가 안 되는 백업은 백업이 아니다.

인수인계

CLAUDE.md, 에이전트 문서, 테스트가 저장소에 있으면 다른 사람도 같은 팀을 이어서 부릴 수 있다.

점검 프롬프트: "운영 전 점검 목록을 만들고 항목마다 시험 결과를 표로 보여 줘"

코드는 팀이 쓰고, 판단은 사람이 한다

이 가이드에서 가져갈 세 가지

1

자료를 모으고 요구사항부터 정리시킨다

양식과 규정을 docs에 넣고, 모르는 것은 추측하지 말고 질문하게 한다.

2

하네스로 팀을 만들고 개발 단계마다 확인한다

작성–검수 구조와 사람의 확인 관문이 품질을 지킨다.

3

피드백을 규칙으로 바꿔 하네스를 깎는다

반복된 지적은 회고로 에이전트 문서에 넣어 다음 프로젝트의 자산으로 만든다.

다음 단계: 우리 조직의 양식 세 장을 docs에 넣고 STEP 1을 시작한다